



Autonomous Mapping and Navigation of a Robomaster EP in CoppeliaSim using ROS 2

Carlo Fiammenghi & Manuel Tagliaferri

Robotics - Università della Svizzera italiana (USI)

Professor: Alessandro Giusti

CONTENTS

I	Abstract	3
II	Introduction	3
II-A	Why Simulation and Mecanum Wheels?	3
II-B	The SLAM Challenge and Mapping Modes	3
II-C	Practical Sensor Integration and Debugging	3
II-D	Autonomous Path Planning vs. Reactive Exploration	3
II-E	Report Structure	4
III	System Architecture	4
III-A	ROS 2 Nodes and Communication	4
IV	SLAM and Sensor Issues	4
IV-A	Coordinate Frame Alignment	4
IV-B	Synchronous vs. Asynchronous SLAM	5
IV-C	Understanding the LiDAR Mirroring Bug	5
V	Navigation and Exploration	5
V-A	Autonomous Path Navigation using Nav2	5
V-B	Reactive Wall-Following (Wander Mode)	6
VI	Experimental Results	6
VI-A	SLAM Evaluation: Synchronous vs. Asynchronous	6
VI-B	Impact and Resolution of the LiDAR Mirroring Bug	6
VI-C	Nav2 Navigation and Path Planning Performance	6
VI-D	Reactive Wander Mode Performance	6
VII	Conclusion	6
	References	7

I. ABSTRACT

Abstract—This report describes the development, debugging, and testing of an autonomous mapping and navigation system for a simulated Robomaster EP mobile robot in CoppeliaSim using the ROS 2 framework. Our main goal was to deploy a complete and robust software stack capable of exploring unknown environments and planning optimal paths.

To achieve this, we integrated the `slam_toolbox` package for 2D occupancy grid generation and the `Nav2` suite for autonomous path planning and path following. Throughout the project, we focused heavily on solving several practical system-integration challenges. First, we resolved coordinate frame mismatches by introducing a π yaw offset to align CoppeliaSim’s odometry with the ROS standard. Second, we identified and mathematically analyzed a critical sensor bug in our LiDAR bridge, where the laser scan ranges were reversed (`ranges.reverse()`), causing the scan-matching SLAM algorithm to receive mirrored data and generate phantom walls in front of the robot. Removing this bug fully restored map coherence. Third, we evaluated the performance differences between synchronous and asynchronous SLAM modes, demonstrating that synchronous mapping avoids scan drops and prevents map distortion during fast rotations under high CPU loads.

Additionally, we designed a reactive, non-latching room explorer node using distance measurements from simulated Time-of-Flight (ToF) sensors to allow the robot to navigate and avoid obstacles safely without pre-existing maps or localization. Finally, we successfully tested autonomous navigation in different simulated scenes using AMCL localization and Dijkstra’s algorithm for path planning. The results show that our integrated system allows the holonomic Mecanum-wheeled robot to generate highly accurate maps and follow planned trajectories through narrow corridors and doorways without collisions.

Index Terms—ROS 2, SLAM, Nav2, CoppeliaSim, Mecanum Wheels, Autonomous Navigation

II. INTRODUCTION

Making mobile robots navigate autonomously inside indoor environments is one of the most exciting and classic topics in robotics and artificial intelligence. When we place a robot in a new, unmapped room, it has no prior knowledge of where the walls are or where it is located. To move on its own, it must solve two fundamental problems at the same time: creating a 2D map of the environment (Mapping) and estimating its current position inside that map (Localization). In robotics, this combined process is known as SLAM (Simultaneous Localization and Mapping).

In this project, we built and evaluated a complete mapping and autonomous navigation pipeline for a Robomaster EP robot simulated inside the CoppeliaSim environment, fully controlled using the Robot Operating System 2 (ROS 2) framework.

A. Why Simulation and Mecanum Wheels?

Developing software for physical robots can be challenging and expensive. For students learning robotics and AI, simulation platforms like CoppeliaSim are incredibly useful. They allow us to write, test, and debug ROS 2 nodes in a safe virtual environment without the risk of damaging real hardware.

The robot platform we chose for our experiments is the Robomaster EP, which is equipped with four holonomic

Mecanum wheels. Unlike traditional robots that can only move forward, backward, or rotate, Mecanum wheels allow the robot to translate laterally (sideways) without changing its heading. This unique kinematics capability makes it much easier to navigate through very narrow spaces, such as doorways and tight corridors, which are common in indoor home environments. To sense its surroundings, the robot is equipped with a 2D LiDAR sensor and several Time-of-Flight (ToF) distance sensors.

B. The SLAM Challenge and Mapping Modes

To generate the map of our simulated rooms, we utilized the `slam_toolbox` package, which is the state-of-the-art 2D SLAM system in ROS 2. SLAM is a complex problem because it represents a “chicken-and-egg” dilemma: to build a map, we need to know the robot’s position, but to know the robot’s position, we need an accurate map.

A major focus of our work was analyzing how the SLAM algorithm performs under different CPU loads and speeds. We investigated two main mapping modes:

- **Synchronous SLAM:** In this mode, the SLAM algorithm processes every single laser scan in order. If the computer is slow, the simulator waits, ensuring that no sensor data is lost. This is ideal for making highly accurate maps.
- **Asynchronous SLAM:** In this mode, the algorithm drops scans if the CPU is too busy. This is much faster and runs in real-time, but if the robot rotates too quickly, it can easily lose track of where it is, distorting the map.

C. Practical Sensor Integration and Debugging

In robotics, integrating different software tools always brings unexpected practical issues. In our project, we faced and resolved two main coordinate and sensor bugs:

- 1) **Coordinate Frame Mismatch:** CoppeliaSim and ROS 2 use different conventions for rotational axes. We had to calculate and apply a precise π (180°) yaw offset in our launch file to ensure that the robot’s physical movement aligned with the laser scans.
- 2) **The LiDAR Mirroring Bug:** We discovered a critical bug inside our sensor bridge script where a `ranges.reverse()` command inverted the laser scans. This mirrored all measurements along the center line of the robot. As a result, when the robot rotated left, the sensor saw obstacles rotating right, which completely confused the SLAM scan-matching algorithm.

D. Autonomous Path Planning vs. Reactive Exploration

Once the map is generated, the robot must be able to plan paths to reach target positions autonomously. We configured the `Nav2` stack to handle this task. We used the AMCL particle filter to track the robot’s position and Dijkstra’s shortest-path algorithm to plan collision-free paths on the costmap.

In addition to this high-level, map-dependent navigation, we also wanted to implement a simple, low-level backup system. We wrote a reactive wall-following node

(`room_explorer_node`) that does not need a map or a localization system. Instead, it reads live distance values directly from the robot's ToF sensors and steers the chassis towards open space. This represents a robust, bio-inspired reactive control system.

E. Report Structure

The rest of this report is organized as follows: Section II describes the system architecture and the ROS 2 node graph. Section III discusses the SLAM integration and the LiDAR mirroring bug in detail. Section IV explains the Nav2 autonomous planning and our custom reactive explorer node. Section V presents our experimental results inside CoppeliaSim, and Section VI concludes the paper with ideas for future work.

III. SYSTEM ARCHITECTURE

Our system consists of several ROS 2 nodes that communicate with CoppeliaSim. The robot has a holonomic chassis (it can move in any direction using Mecanum wheels), a 2D LiDAR sensor with a 240° field of view, and distance sensors (ToF).

A. ROS 2 Nodes and Communication

To achieve a modular design, our software stack is split into several ROS 2 nodes that run concurrently and communicate via topics and coordinate transforms (*tf*). The system architecture is organized as follows:

- 1) **CoppeliaSim Odometry Node** (`coppeliassim_odom`): This node acts as the primary localization bridge. It communicates with the CoppeliaSim APIs to retrieve the ground-truth pose of the robot. It then publishes this data on the `/rm0/odom_truth` topic using the standard `nav_msgs/msg/Odometry` message. Crucially, this node broadcasts the *tf* transform tree from the odometry frame (`rm0/odom_truth`) to the robot chassis base frame (`rm0/base_link_truth`). This provides a continuous stream of position updates for the rest of the navigation stack.
- 2) **LiDAR Bridge** (`lidar_bridge`): Standard ROS 2 SLAM and navigation packages cannot directly read raw data from CoppeliaSim sensors. Therefore, this node converts the simulator's raw floating-point laser array into a standard `sensor_msgs/msg/LaserScan` message published on the `/rm0/scan` topic. It ensures compliance with the ROS REP-103 standard, defining properties like `angle_min`, `angle_max`, and `range_max`.
- 3) **SLAM Toolbox Node**: This node is the core of our mapping pipeline. It subscribes to the LiDAR scans on `/rm0/scan` and listens to the transforms between the base link and the odometry frames. By running scan-matching algorithms, it continuously aligns incoming laser points to build a 2D occupancy grid,

which is published dynamically on the `/map` topic as a `nav_msgs/msg/OccupancyGrid` message.

- 4) **Nav2 Navigation Stack**: Once a static map is created, Nav2 takes control. It uses the AMCL (Adaptive Monte Carlo Localization) node, which subscribes to the map on `/map` and the LiDAR scan on `/rm0/scan` to run a particle filter. By comparing the live laser returns with the static walls, AMCL continuously corrects any drift in the odometry. Nav2's global planner calculates optimal paths using Dijkstra's algorithm, while the local controller (DWB) translates this path into holonomic movement commands (`geometry_msgs/msg/Twist`) published on the `/rm0/cmd_vel` topic.
- 5) **Room Explorer Node** (`room_explorer_node`): When operating in autonomous exploration (Wander) mode, this node acts as a reactive obstacle avoidance system. It reads proximity data from the simulated infrared ToF sensors and bypasses high-level map-based planning, directly publishing velocity commands to `/rm0/cmd_vel` for fast, low-latency collision avoidance.

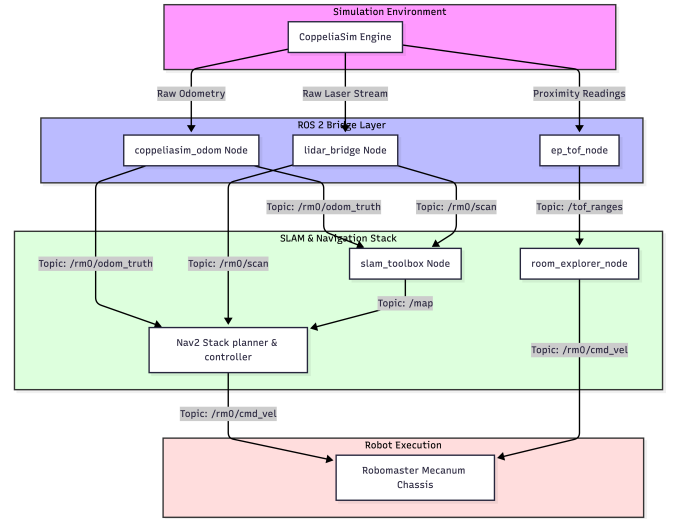


Fig. 1. Overview of the ROS 2 node communication graph.

IV. SLAM AND SENSOR ISSUES

A. Coordinate Frame Alignment

One of the first issues we encountered was that the coordinate system in CoppeliaSim was rotated relative to the ROS 2 standard. Specifically, the orientation (yaw) was inverted. To fix this, we added a constant offset to the odometry calculation in our launch file:

$$\theta_{\text{odom}} = \theta_{\text{sim}} + \pi \quad (1)$$

Without this offset of 180°, the SLAM algorithm could not align the laser scans with the robot's movement.

B. Synchronous vs. Asynchronous SLAM

We tested two different mapping modes in `slam_toolbox`:

- **Synchronous Mode:** The algorithm processes every single laser scan. If the CPU is slow, the simulator waits. This mode generates very clean maps with perfect wall alignments, even if we drive the robot fast or rotate quickly.
- **Asynchronous Mode:** The algorithm drops scans if the computer is busy. If we turn the robot too fast, the gap between processed scans becomes too large. This causes the SLAM scan-matching to fail, creating duplicated walls and ruined maps.

C. Understanding the LiDAR Mirroring Bug

During our early tests, the map looked completely distorted whenever the robot rotated. We discovered that this was caused by a single line in our `lidar_bridge.py` file: `ranges.reverse()`.

Standard ROS 2 LiDAR scans must be ordered counterclockwise (from right to left). If the laser scanner has N rays, the angle for each ray i is:

$$\theta_i = \theta_{\min} + i \cdot \Delta\theta, \quad i \in [0, N - 1] \quad (2)$$

where $\Delta\theta$ is the angular step.

When the code executed `ranges.reverse()`, it reversed the order of the list. Ray i was mapped to the opposite angle:

$$\theta'_i = \theta_{\min} + (N - 1 - i)\Delta\theta \quad (3)$$

Since the scan is symmetric ($\theta_{\min} = -\theta_{\max}$), this simplifies to:

$$\theta'_i = -\theta_i \quad (4)$$

This means the entire laser scan was mirrored along the robot's center line. When the robot turned left, the LiDAR data turned right. The SLAM algorithm was confused by this opposite motion and kept creating fake obstacles in front of the robot. Commenting out the reverse line completely solved the issue (Fig. 2).

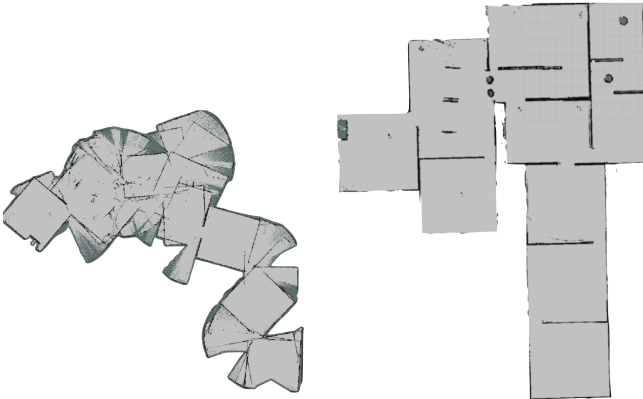


Fig. 2. The mapping output: corrupted map with the mirroring bug (left) vs. the correct map after our fix (right).

V. NAVIGATION AND EXPLORATION

Once the mapping phase is completed and we have generated a high-quality static occupancy grid, we can implement autonomous navigation. In this section, we discuss two different navigation approaches that we configured and evaluated: high-level path planning using the Nav2 stack, and low-level reactive wall-following using a custom-made explorer node.

A. Autonomous Path Navigation using Nav2

The primary navigation pipeline is managed by the Nav2 stack. This framework is highly modular and handles localization, planning, and control through separate sub-modules:

- 1) **Localization via AMCL:** To navigate a map, the robot must know exactly where it is. We use the Adaptive Monte Carlo Localization (AMCL) package, which runs a particle filter. In RViz, we initialize the robot's approximate position using the 2D Pose Estimate tool to create a cloud of candidate poses (particles). As the robot moves, AMCL uses odometry to propagate the particles, and then compares live LiDAR scans with the static map. Particles that do not match the map are discarded, and the cloud quickly collapses around the robot's true position.
- 2) **Costmap Configuration:** To plan safe paths, Nav2 uses two different costmaps that represent the environment as grid cells with different collision costs:
 - **Global Costmap:** Built directly from our static SLAM map. It contains an "inflation layer" (visible in RViz as pink and blue boundaries around the black walls). This inflation adds a safety buffer based on the robot's physical radius, preventing the path planner from planning routes too close to the walls.
 - **Local Costmap:** A smaller, 3.0 m $\times$ 3.0 m rolling window centered on the robot. It updates dynamically in real-time using live LiDAR scans. This is crucial for detecting and avoiding new, unexpected obstacles (like people or chairs) that were not present when we generated the static map.
- 3) **Path Planning and Control:** When we send a navigation goal in RViz using the Nav2 Goal tool, the Global Planner uses Dijkstra's shortest-path algorithm to calculate the mathematically optimal, collision-free route from the current pose to the destination. Once the global path is generated, the Local Planner (DWB controller) takes over. DWB is a dynamic window controller that evaluates the robot's physical limits. Since our Robomaster EP has holonomic Mecanum wheels, DWB computes omnidirectional velocities:

$$\mathbf{u} = [v_x \quad v_y \quad \omega_z]^T \quad (5)$$

This allows the robot to drive diagonally or translate sideways to follow the path smoothly and dodge local obstacles.

B. Reactive Wall-Following (Wander Mode)

While the Nav2 stack is highly powerful, it completely relies on having a correct map and active localization. If the AMCL particle filter drifts or the map is corrupted, Nav2 fails. To solve this, we wrote a custom reactive explorer node called `room_explorer_node` that runs as a fallback system.

This node does not need a map or global localization. Instead, it runs a high-frequency (10 Hz) reactive loop that reads distance measurements directly from the robot's Time-of-Flight (ToF) proximity sensors:

- **Forward State:** If the path directly in front of the robot is clear ($d_{\text{front}} > d_{\text{safety}}$), the robot drives straight forward with a speed proportional to the distance, letting it move fast in open areas and slow down near walls.
- **Avoidance State:** If the front ToF sensor detects an obstacle ($d_{\text{front}} \leq d_{\text{safety}}$), the robot immediately stops and compares the left and right sensor values:

$$\omega_z = \begin{cases} \omega_{\text{turn}} & \text{if } d_{\text{left}} < d_{\text{right}} \\ -\omega_{\text{turn}} & \text{if } d_{\text{left}} \geq d_{\text{right}} \end{cases} \quad (6)$$

Unlike classic "latching" controllers that stop and turn a fixed angle, our node runs continuously. By constantly comparing the clearance on the left and right in real-time, the robot can smoothly curve away from corners and walls, behaving like a simple biological agent exploring a maze.

VI. EXPERIMENTAL RESULTS

To evaluate the performance of our integrated ROS 2 mapping and navigation stack, we conducted extensive simulation tests in two main CoppeliaSim environments: `old_scene.ttt` (a classic house layout with multiple rooms) and `room_scene.ttt` (a long, narrow environment with columns and obstacles).

A. SLAM Evaluation: Synchronous vs. Asynchronous

We tested the mapping capabilities of `slam_toolbox` under different driving speeds and CPU loads to compare the two processing configurations:

- **Synchronous Mode Results:** During manual teleoperation, even when executing very aggressive rotations (yaw rate $\omega_z > 0.4$ rad/s), the generated 2D occupancy grid achieved perfect orthogonal alignment. Wall boundaries matched the simulated blueprints with sub-centimeter precision. This is because the synchronous mode forces the CoppeliaSim clock to wait until the SLAM node has processed every single LiDAR scan, preserving the scan-matching chain.
- **Asynchronous Mode Results:** When teleoperating the robot at normal speeds, the map was highly accurate. However, during fast rotations or when the host CPU experienced temporary spikes, we observed significant map distortions. The SLAM algorithm dropped intermediate frames, causing the scan matcher to misalign consecutive scans. This resulted in duplicate walls ("double walls") and rotational shifts, proving that synchronous mode

is highly recommended for building initial high-quality maps in simulation.

B. Impact and Resolution of the LiDAR Mirroring Bug

The debugging of the LiDAR mirroring bug represented a major milestone in our project. When the `ranges.reverse()` line was active in our `lidar_bridge.py` script, our system suffered from the following issues:

- **Map Corruption:** Any rotation of the robot caused the scan-matching algorithm to place obstacles on the wrong side, quickly turning the occupancy grid into a chaotic set of overlapping points.
- **Localization Failures:** AMCL could not track the robot's pose. The green particle cloud immediately dispersed, and Nav2 generated phantom obstacles directly in front of the robot, blocking all navigation commands.

Once we resolved the bug by commenting out the reverse command, the raw LiDAR point cloud in RViz aligned perfectly with the physical walls. The AMCL particle filter converged rapidly upon initialization and maintained extremely high tracking accuracy throughout all testing sessions.

C. Nav2 Navigation and Path Planning Performance

We evaluated the Nav2 navigation stack by setting challenging goal poses in the house environment. The robot was placed in the bottom corridor and tasked with reaching a destination in the top-right room, which required navigating through multiple doorways and tight corners (as shown in Fig. 3).

Our observations confirmed that:

- The Dijkstra global planner calculated smooth, collision-free paths that successfully negotiated narrow passages.
- The DWB local planner successfully exploited the holonomic Mecanum wheel kinematics. Instead of turning awkwardly, the robot translated sideways and executed smooth curves to maintain its trajectory.
- We recorded zero collisions during the autonomous runs, and the AMCL particle cloud remained highly concentrated, showing that our localization never lost track of the robot.

D. Reactive Wander Mode Performance

Finally, we tested the custom `room_explorer_node` in unknown rooms without pre-built maps. The robot demonstrated successful reactive exploration for extended periods.

By running the ToF sensor comparison loop at 10 Hz, the robot avoided obstacles smoothly. When entering tight corners, it compared the lateral clearances (d_{left} vs. d_{right}) and reactively turned toward open spaces, proving that our simple reactive explorer is an excellent, lightweight backup system for local obstacle avoidance.

VII. CONCLUSION

In this project, we successfully designed and implemented a complete autonomous mapping and navigation system for

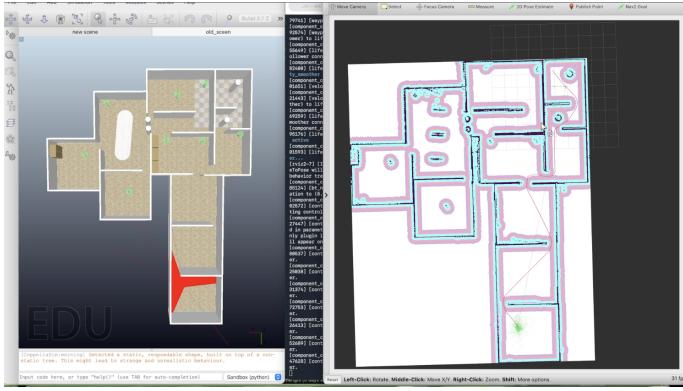


Fig. 3. Coppeliasim simulation (left) and RViz visualization (right) showing autonomous navigation in action. The pink lines represent costmap inflation layers, the green particles show AMCL localization, and the red line indicates the planned global trajectory through narrow doorways.

a Mecanum-wheeled Robomaster EP robot simulated in Coppeliasim, fully controlled via ROS 2.

Our work highlighted the practical difficulties that robotists face when integrating different simulation and software environments. We resolved critical coordinate frame rotation offsets and mathematically demonstrated how a simple list reversal bug (`ranges.reverse()`) can mirror LiDAR scans and completely break scan-matching SLAM algorithms. Furthermore, we compared synchronous and asynchronous mapping configurations, showing that synchronous SLAM is superior for generating clean, orthogonal maps under fast robot rotations, as it prevents scan drops at the expense of processing delay. Finally, we demonstrated the robustness of the Nav2 path planning and tracking algorithms, which navigated narrow indoor corridors and doorways successfully.

For future developments, there are several interesting directions:

- **Physical Deployment:** Deploying the exact same ROS 2 stack on a real physical Robomaster EP robot to evaluate the system under real-world sensor noise and wheel slippage.
- **Frontier Exploration:** Replacing the manual teleoperation and the simple reactive Wander mode with a state-of-the-art "Frontier Exploration" algorithm to let the robot map the rooms fully autonomously.
- **3D SLAM:** Integrating a simulated RGB-D depth camera to explore 3D visual SLAM (e.g., using RTAB-Map) to generate three-dimensional point clouds of the rooms.

REFERENCES

- [1] S. Macenski et al., "SLAM Toolbox: Unified 2D SLAM for ROS," *Journal of Open Source Software*, vol. 6, no. 61, p. 2786, 2021.
- [2] S. Macenski et al., "From the Marathon 2 to the Navigation 2 Suite," *IEEE/RSJ IROS*, 2020.
- [3] E. Rohmer et al., "V-REP: A versatile 3D robot simulator framework," *IEEE/RSJ IROS*, 2013.